Смоленский колледж телекоммуникаций (филиал)
федерального государственного бюджетного образовательного учреждения
высшего образования
«Санкт-Петербургский государственный университет телекоммуникаций
им. проф. М. А. Бонч-Бруевича»

КУРСОВОЙ ПРОЕКТ

по МДК 02.01 «Технология разработки программного обеспечения»
специальность 09.02.07 «Информационные системы и программирование»

Тема: «Разработка мобильного приложения для бронирования загородных особняков и VIP-услуг «ВардКлик»»

Выполнил: студент 4 курса
группы ИСП823
Федоров Матвей Валерьевич
_____ (подпись)

Руководитель: преподаватель
Зимина Е.А.
_____ (подпись)

Оценка: _____
Дата защиты: «___» _____ 2026 г.

Смоленск 2026

УТВЕРЖДАЮ

Зав. учебной частью

_____ Дроздович С.Н.

«___»_____2026г.

ЗАДАНИЕ

на курсовой проект

По МДК: 02.01 Технология разработки программного обеспечения

Студенту: 4 курса ИСП823 группы

Федорова Матвея Валерьевича

(фамилия, имя, отчество)

Тема проекта: Разработка мобильного приложения для бронирования загородных особняков и VIP-услуг «ВардКлик»

Исходные данные: Язык программирования Java, Kotlin, среда разработки — Android Studio, платформа разработки — Android (Android SDK), система управления базами данных MS SQL Server Management Studio, направление разработка мобильного приложения для бронирования загородных особняков и VIP-услуг «ВардКлик»

В выполненном курсовом проекте должны быть представлены:

1.ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

Введение

1. Теоретический раздел

2. Практический раздел

3. Охрана труда

Заключение

Список источников информации

Приложение

2.ГРАФИЧЕСКАЯ ЧАСТЬ РАБОТЫ

Лист: 1. Диаграмма БД

2. Диаграмма классов

3. Диаграмма состояний

4. Общий алгоритм работы приложения в виде блок-схемы

Дата выдачи: «___»_____2026 г. Срок окончания: «___»_____2026 г.

Преподаватель – руководитель курсового проекта_____ Зимина Е.А.

(подпись) ФИО

Председатель методической комиссии: _____ Строде Т.Н.

(подпись) ФИО

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....4
1. ТЕОРЕТИЧЕСКИЙ РАЗДЕЛ.....6
1.1 Анализ предметной области, исходной проектной и технической документации.....6
1.2 Функциональные и нефункциональные требования к программному продукту.....8
1.3 Требования к отдельным программным модулям, их входным и выходным данным.....9
1.4 Описание структуры продукта и взаимодействия компонентов.....10
1.5 Обоснование выбора языка программирования, среды разработки, библиотек и других инструментов.....12
1.6 Принятые стандарты кодирования и план тестирования.....13
2. ПРАКТИЧЕСКИЙ РАЗДЕЛ.....14
2.1 Требования к аппаратному и программному обеспечению.....14
2.2 Описание пользовательского интерфейса, основных алгоритмов и структур данных.....14
2.3 Реализация программных модулей и описание их интеграции.....17
2.4 Обработка входных данных, исключительных ситуаций и ошибок.....18
2.5 Разработка тестовых наборов и тестовых сценариев.....19
2.6 Анализ результатов тестирования и описание исправленных дефектов....19
2.7 Инспектирование компонентов программы по чек-листу.....20
2.8 Краткая инструкция пользователя.....20
3. ОХРАНА ТРУДА.....21
3.1 Опасные и вредные производственные факторы.....21
3.2 Требования к организации рабочего места и освещению.....21
3.3 Режим труда и отдыха.....22
3.4 Безопасная эксплуатация оборудования.....22
ЗАКЛЮЧЕНИЕ.....23
СПИСОК ИСТОЧНИКОВ ИНФОРМАЦИИ.....24
ПРИЛОЖЕНИЕ А. Экранные формы приложения.....25
ПРИЛОЖЕНИЕ Б. Фрагменты исходного кода.....28
ПРИЛОЖЕНИЕ В. Чек-лист инспектирования кода.....30

ВВЕДЕНИЕ

Загородный отдых в особняке всё чаще становится не просто арендой жилья, а комплексной услугой. Особняки, виллы, усадьбы и шале снимают для семейных праздников, корпоративных мероприятий и отдыха большой компанией, нередко сразу несколько домов, и вместе с жильём гостям нужны личный водитель, шеф-повар, горничная, охрана, а иногда и вертолёт до места отдыха. Сейчас такие услуги приходится искать и заказывать по отдельности у разных исполнителей, а их цены заранее неизвестны. Потребность в едином мобильном приложении, в котором можно выбрать город, подобрать один или несколько особняков и сразу заказать к ним VIP-услуги с понятной итоговой стоимостью, и определила выбор темы курсового проекта.

Спрос на внутренний туризм и загородный отдых в России устойчиво растёт, а смартфон стал для пользователя основным инструментом поиска и бронирования. Универсальные сервисы аренды жилья слабо учитывают особенности загородных особняков: большую вместимость, бассейн, баню, охрану, вертолётную площадку, подтверждение каждой заявки администратором, а также то, что стоимость одних и тех же услуг в элитном особняке и в загородном шале не может быть одинаковой. Поэтому разработка специализированного приложения «ВардКлик», которое объединяет каталог особняков, карту, заказ VIP-услуг, собственную систему ценообразования и скидки, является актуальной. С технической стороны проект решает типовые задачи мобильной разработки: поиск и фильтрацию объектов, проверку занятости дат, расчёт стоимости заказа и обмен данными между Android-приложением и сервером с базой данных MS SQL Server.

Целью курсового проекта является проектирование, программная реализация и тестирование мобильного приложения для бронирования загородных особняков и VIP-услуг «ВардКлик» для операционной системы Android.

Для проектирования выбран объектно-ориентированный подход с моделированием на языке UML. Он подходит проекту, потому что предметная область естественно описывается объектами с собственными данными и поведением: особняк с ценой за ночь и коэффициентом класса, аренда, которая проходит статусы «ожидает подтверждения», «подтверждена», «завершена» или «отменена», заказ, объединяющий несколько аренд, и VIP-услуга, цена которой зависит от особняка. Диаграмма классов фиксирует эти сущности и связи между ними до написания кода, диаграмма состояний — действия пользователя и смену статусов аренды, диаграмма базы данных — таблицы MS SQL Server, а блок-схема — общий алгоритм работы приложения.

Программная архитектура строится по шаблону MVVM (Model–View–ViewModel), который компания Google рекомендует для приложений на Jetpack Compose. Для «ВардКлик» он важен по двум причинам: состояние экранов (выбранные даты, услуги, рассчитанная стоимость) хранится в модели представления и не теряется при повороте экрана, а слой данных отделён от интерфейса, поэтому источник данных — демонстрационный сервер на устройстве или REST-сервер с базой MS SQL Server — меняется одним флагом сборки без изменения экранов.

Методологией разработки выбрана итеративная (инкрементная) модель жизненного цикла. Она соответствует проекту, так как требования уточнялись по ходу работы: после первой версии каталога понадобились карта и фильтры, после бронирования — VIP-услуги с ценами по классу особняка, затем скидки и общий заказ на несколько особняков. Каскадная модель потребовала бы зафиксировать все требования заранее, а гибкие методологии вроде Scrum рассчитаны на команду, тогда как проект выполняется одним разработчиком. Работа велась в пять итераций: каталог и карта; бронирование с проверкой дат; VIP-услуги и ценообразование; общий заказ и панель администратора; оформление интерфейса и тестирование. Каждая итерация завершалась рабочей сборкой APK, которая собиралась в GitHub Actions и проверялась на эмуляторе Android.

Для достижения цели необходимо решить следующие задачи:

1. провести анализ предметной области и существующих сервисов аренды загородного жилья;
2. сформулировать требования к приложению и спроектировать его архитектуру, диаграммы классов и состояний;
3. спроектировать базу данных: особняки, города, аренды, заказы, VIP-услуги, отзывы и пользователи;
4. реализовать пользовательский интерфейс: каталог особняков с фильтрами и картой, карточку особняка, оформление аренды и заказа, личные разделы и панель администратора;
5. реализовать бизнес-логику: проверку занятости дат, цены VIP-услуг по классу особняка, скидки за пакет «Всё включено» и за заказ нескольких особняков;
6. разработать модуль взаимодействия с базой данных и провести тестирование приложения;
7. рассмотреть вопросы охраны труда и организации безопасного рабочего места программиста.

1. ТЕОРЕТИЧЕСКИЙ РАЗДЕЛ

1.1 Анализ предметной области, исходной проектной и технической документации

Предметной областью курсового проекта является посуточная аренда загородной недвижимости премиального класса — особняков, вилл, усадеб и шале — вместе с дополнительными услугами для гостей. В отличие от аренды городской квартиры, аренду особняка обычно связывают с событием: семейным праздником, свадьбой, юбилеем, корпоративным выездом или отдыхом большой компании. Спрос на такие объекты сезонный — пики приходятся на летние месяцы и новогодние праздники, — а сами объекты сосредоточены в пригородах крупных городов и в курортных регионах.

Сегодня аренда особняка организуется в основном вручную. Клиент находит объект на сайте агентства, в общих досках объявлений или по рекомендации, связывается с владельцем или управляющим по телефону или в мессенджере и уточняет свободные даты. Владелец ведёт график занятости в тетради или электронной таблице, поэтому ответ приходит не сразу, а при нескольких одновременных запросах возможно двойное бронирование. После согласования клиент вносит предоплату, подписывает договор аренды и оставляет залог, а при заезде и выезде составляется акт приёма-передачи. Дополнительные услуги — трансфер, повар, уборку, охрану, перелёт на вертолёте — клиент заказывает отдельно у разных исполнителей, каждый со своей ценой и своими условиями.

Участниками процесса являются арендатор (частное лицо или организатор мероприятия), владелец или управляющий объектом, агентство или администратор площадки, который ведёт каталог и проверяет объекты, и исполнители дополнительных услуг. Основные информационные объекты процесса — объект недвижимости с характеристиками (вместимость, число спален, удобства, наличие вертолётной площадки), график занятости, прайс-лист на проживание и услуги, заявка на бронирование, договор и отзывы гостей.

Анализ процесса выявил его основные проблемы: нет единого источника сведений о свободных датах, поэтому подтверждение занимает время и возможны накладки; стоимость дополнительных услуг непрозрачна и не связана с классом объекта, хотя одна и та же услуга в элитном особняке и в загородном шале стоит по-разному; для аренды нескольких домов на одно мероприятие приходится договариваться с каждым владельцем отдельно; отзывы о проживании разрознены. Эти проблемы и обосновывают выбор темы: их решает мобильное приложение с единым каталогом, автоматической проверкой занятости, расчётом цены услуг по классу особняка и общим заказом на

несколько объектов.

Исходной проектной документацией служит задание на курсовой проект. В нём определены тема, исходные данные — языки программирования Java и Kotlin, среда разработки Android Studio, платформа Android (Android SDK), СУБД MS SQL Server с SQL Server Management Studio, — а также состав пояснительной записки и графической части: диаграмма базы данных, диаграммы классов и состояний и блок-схема общего алгоритма. Технической документацией при проектировании служили официальные руководства Android Developers по Jetpack Compose и архитектуре приложений, документация языка Kotlin, Yandex MapKit и Microsoft SQL Server.

Для обоснования темы рассмотрены сервисы, которые применяются для поиска и бронирования жилья в России, и международная платформа Airbnb.

Airbnb — международная платформа аренды жилья у частных владельцев с поиском по датам и гостям, картой, подробными карточками, отзывами и избранным; её интерфейс считается эталонным для подобных приложений. Однако работа сервиса в России приостановлена с 2022 года, особняки представлены наравне с квартирами и комнатами, дополнительные услуги не заказываются вместе с жильём, а единого заказа на несколько объектов нет.

Суточно.ру — российский сервис посуточной аренды квартир, домов и коттеджей с поиском по городу и датам, фильтрами, картой и отзывами; бронирование подтверждает сам владелец. Сервис ориентирован на массовый сегмент: премиальные особняки не выделены, VIP-услуги отсутствуют, а качество описаний и подтверждения зависит от конкретного владельца.

Островок — сервис онлайн-бронирования гостиниц и апартаментов с большим каталогом, мгновенным подтверждением и программами лояльности. Его фокус — гостиничные номера и деловые поездки: аренда целого особняка для мероприятия, заказ водителя, повара или охраны и цены услуг по классу объекта не предусмотрены.

Яндекс Путешествия — сервис бронирования отелей и билетов в экосистеме Яндекса с удобным поиском и интеграцией с Яндекс Картами. Он также ориентирован на отели: загородные особняки встречаются эпизодически, VIP-услуг и единого заказа на несколько объектов нет.

Авито Путешествия — направление площадки Авито для посуточной аренды квартир и домов с большой базой объявлений, картой, отзывами и избранным. Объявления размещают частные владельцы, премиальный сегмент не выделен, отсутствуют VIP-услуги, пакетные скидки и проверка заявок администратором.

Таким образом, ни один из рассмотренных сервисов не объединяет аренду премиального загородного особняка с заказом VIP-услуг, ценами услуг по классу

объекта и единым заказом на несколько особняков.

1.2 Функциональные и нефункциональные требования к программному продукту

Функциональные требования описывают основные действия пользователя в приложении, а нефункциональные — что должна выполнить программа в ответ на каждое из этих действий. Требования приведены в таблице 1.

Таблица 1 – Функциональные и нефункциональные требования

№	Действие пользователя (функциональное требование)	Реакция программы (нефункциональное требование)
1	Выбрать город на главном экране	загрузить и показать особняки города за 2–3 секунды; при отсутствии сети — сообщение и кнопка «Повторить»
2	Найти особняк по названию или адресу, задать фильтры (тип, гости, даты, удобства) и сортировку	отфильтровать список без перезагрузки экрана; при пустом результате — сообщение «ничего не найдено»
3	Переключиться со списка на карту	показать маркеры особняков с ценой за ночь на Яндекс Карте; без ключа карты — подсказку вместо карты
4	Открыть карточку особняка	показать фото, удобства, мини-карту, отзывы и цены VIP-услуг, рассчитанные по классу особняка
5	Зарегистрироваться или войти	проверить формат email и длину пароля (не менее 6 символов), сохранить сессию между запусками
6	Выбрать даты аренды и число гостей	не дать выбрать прошедшие и занятые даты, срок больше 30 ночей и гостей больше вместимости
7	Выбрать VIP-услуги или пакет «Всё включено»	сразу пересчитать стоимость: скидка 15 % на услуги и 5 % на проживание; «вертолёт» — только при наличии площадки
8	Оформить аренду или добавить особняк в общий заказ	повторно проверить даты на сервере, сохранить аренду в статусе «ожидает подтверждения»; в заказе из двух и более особняков — скидка 7 % на проживание
9	Просмотреть свои аренды и отменить предстоящую	разделить аренды по вкладкам; статус «завершено» определить по дате выезда; запретить отмену завершённой аренды
10	Оставить отзыв о проживании	принять один отзыв только о завершённой аренде и пересчитать рейтинг особняка по отзывам
11	Администратор: подтвердить или отменить аренду, добавить или скрыть особняк	открыть панель только для роли «администратор»; на сервере проверять роль при каждом запросе (ошибка 403)

Общие нефункциональные требования: работа на устройствах с Android 8.0 и новее, русскоязычный интерфейс со светлой и тёмной темами, ожидание ответа сервера не более 10 секунд с понятным сообщением об ошибке, хранение ключа Яндекс Карт вне исходного кода. Для запуска проекта нужны Android Studio с Android SDK (API 37) и эмулятор или смартфон с Android 8.0 и новее; по умолчанию приложение работает в демонстрационном режиме без сервера. Тестовые учётные записи: user@wardclick.ru и admin@wardclick.ru, пароль 123456.

1.3 Требования к отдельным программным модулям, их входным и выходным данным

Приложение разделено на программные модули по функциональному назначению: модуль включает экран, модель представления ViewModel и репозиторий для обращения к серверу. Назначение модулей, их входные и выходные данные приведены в таблице 2.

Таблица 2 – Программные модули приложения

Модуль	Назначение и требования	Входные данные	Выходные данные
Авторизация	вход и регистрация, проверка email и пароля, сохранение сессии	email, пароль, ФИО, телефон	токен, идентификатор и роль пользователя
Каталог и поиск	выбор города, поиск, фильтры, сортировка, список и карта	город, строка поиска, даты, число гостей, фильтры	список особняков, маркеры на карте
Карточка особняка	описание, удобства, цены VIP-услуг, отзывы, мини-карта, избранное	идентификатор особняка	данные особняка, отзывы, цены услуг
Аренда и заказ	выбор дат, гостей и услуг, расчёт стоимости, заказ из нескольких особняков	даты заезда и выезда, число гостей, выбранные услуги	аренды в статусе «ожидает подтверждения»
Аренды	аренды по статусам, отмена, отзыв о завершённом проживании	идентификатор аренды, оценка 1–5, текст отзыва	списки аренд, отзыв, новый рейтинг
Администрирование	статистика, подтверждение и отмена аренд, добавление и скрытие особняков	токен администратора, данные особняка	статистика, новые статусы аренд, новый особняк
Ценообразование	цена услуги по коэффициенту особняка, скидки «Всё включено» и за несколько особняков	цена за ночь, число ночей, базовые цены услуг, коэффициент класса	стоимость проживания и услуг, скидка, итог

Сущности, с которыми работают модули, и связи между ними показаны на диаграмме классов (рисунок 1). Пользователь оформляет аренды и заказы, заказ состоит из нескольких аренд, каждая аренда относится к одному особняку, может включать несколько VIP-услуг, а после завершения — один отзыв. Особняк расположен в городе и имеет набор удобств. Перечисление СтатусАренды задаёт допустимые статусы аренды, а класс Ценообразование рассчитывает цены услуг по классу особняка и скидки. Для атрибутов указаны типы данных, для методов — типы возвращаемых значений; типы согласованы с базой данных: идентификаторы — целые числа (INT IDENTITY), денежные суммы и коэффициенты — DECIMAL.

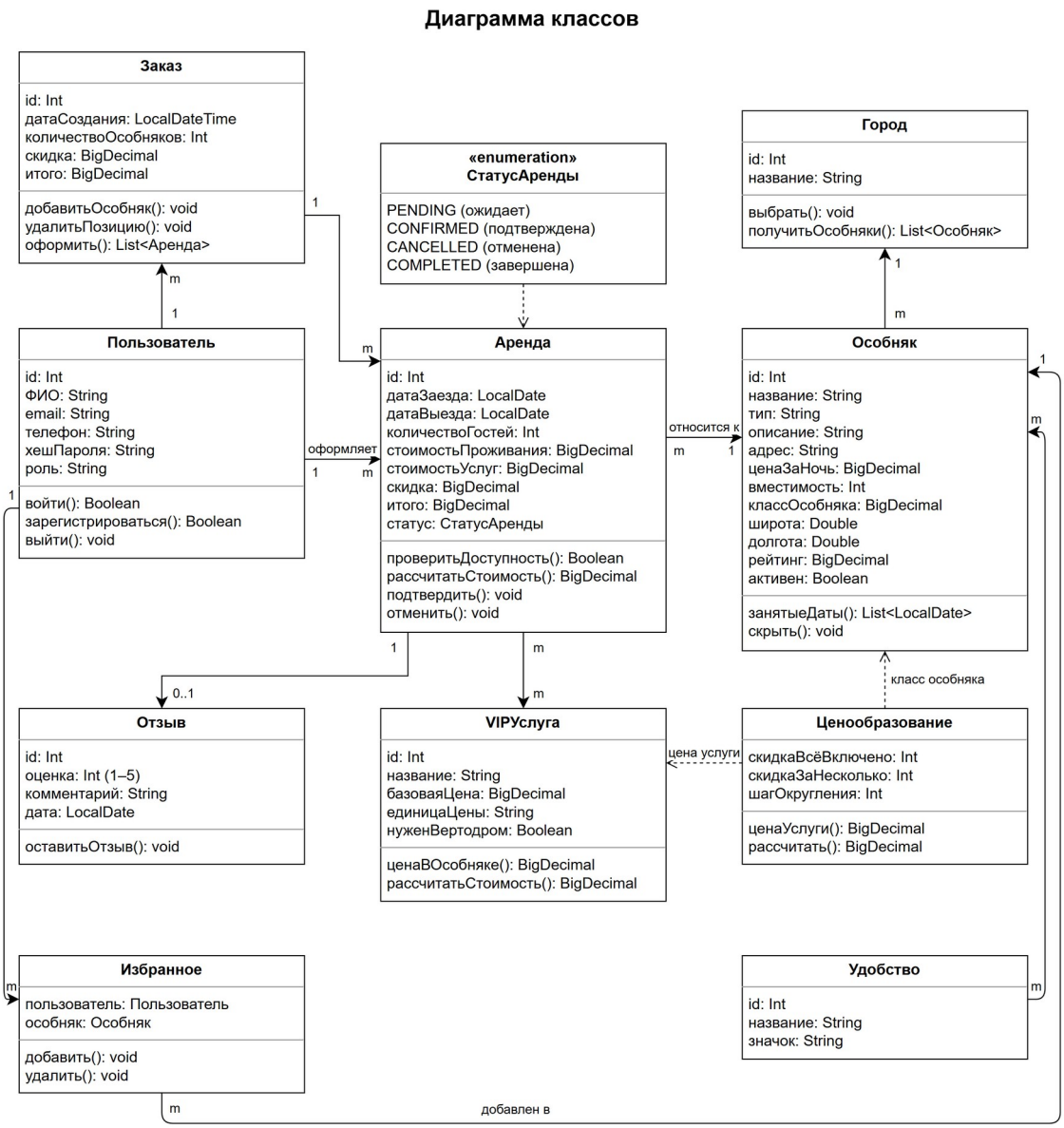


Рисунок 1 – Диаграмма классов

1.4 Описание структуры продукта и взаимодействия компонентов

Приложение построено по шаблону MVVM и состоит из трёх слоёв: слоя представления (экраны на Jetpack Compose, компоненты, тема и навигация), слоя моделей представления (состояние экранов в StateFlow) и слоя данных (репозитории, интерфейс ApiService с реализациями для REST-сервера и демонстрационного режима, модели данных, хранилище сессии и правила ценообразования). Компоненты взаимодействуют в одном направлении: экран передаёт событие в ViewModel, она вызывает функцию репозитория, репозиторий обращается к ApiService и возвращает результат, после чего ViewModel обновляет состояние и экран перерисовывается. Зависимости создаёт контейнер AppContainer, который выбирает реализацию ApiService по флагу сборки USE_MOCK.

Поведение приложения описывает диаграмма состояний (рисунок 2),

разделённая на дорожки «Просмотр и вход», «Оформление аренды», «Обработка аренды» и «Завершение». Приложение открывается на главном экране с выбором города, пользователь без входа переходит в каталог и к карточке особняка. Вход проверяется только при нажатии «Арендовать»: если вход не выполнен, пользователь вводит email и пароль, а при отсутствии аккаунта сначала регистрируется. Далее следуют выбор свободных дат, VIP-услуг и расчёт стоимости; аренда подтверждается сразу или добавляется в общий заказ. Если даты оказались заняты, выводится сообщение об ошибке и пользователь возвращается к выбору дат. Сохранённая аренда получает статус «ожидает подтверждения», после решения администратора — «подтверждено» или «отменено», а после даты выезда — «завершено», и пользователь может оставить отзыв.

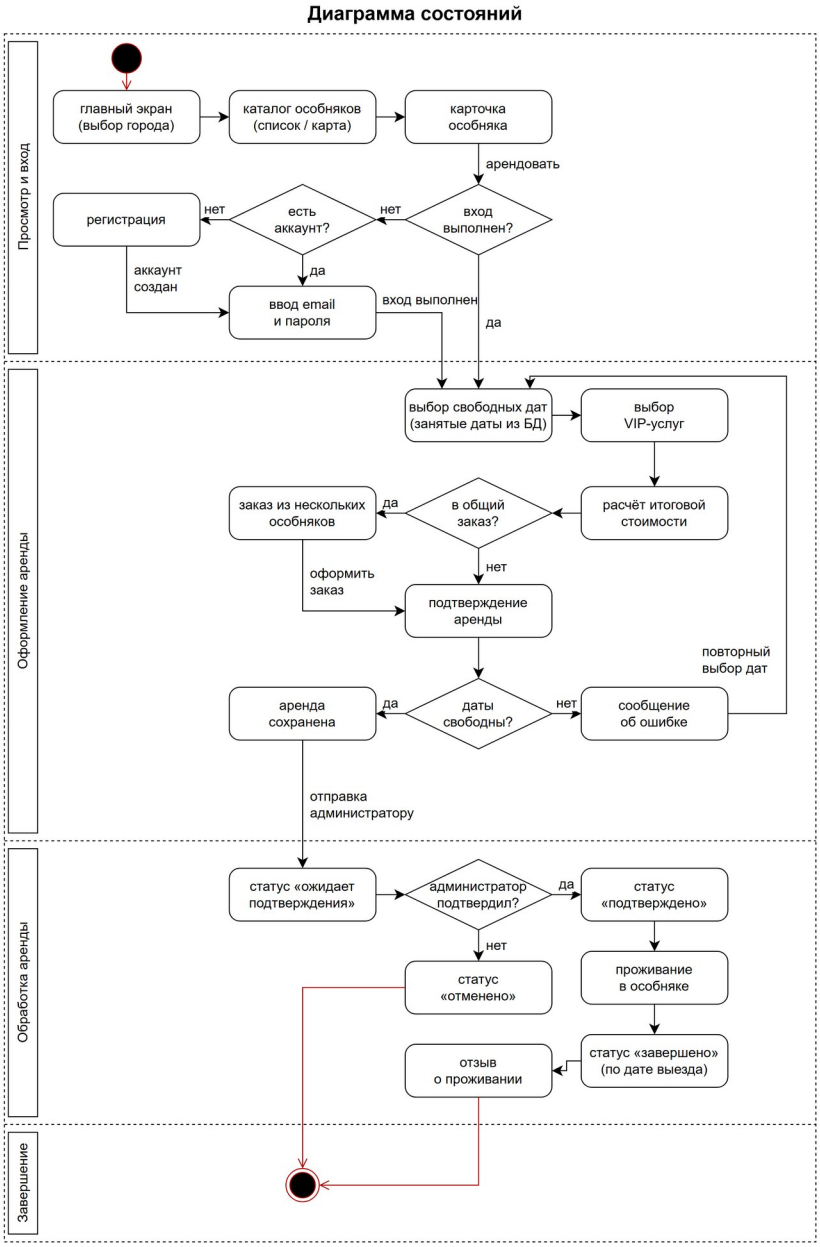


Рисунок 2 – Диаграмма состояний

1.5 Обоснование выбора языка программирования, среды разработки, библиотек и других инструментов

Исходные данные задания — языки Java и Kotlin, среда Android Studio, платформа Android (Android SDK) и СУБД MS SQL Server с SQL Server Management Studio. Выбор каждого инструмента обоснован сравнением с распространёнными альтернативами (таблицы 3–6).

Таблица 3 – Сравнение языков программирования для мобильной разработки

Критерий	Kotlin	Java	Dart (Flutter)	JavaScript (React Native)
Статус для Android	основной язык, рекомендуемый Google	поддерживается Google	кроссплатформенный фреймворк Google	кроссплатформенный фреймворк Meta
Доступ к Android SDK и Jetpack Compose	полный, напрямую	SDK — полный, Compose — нет	через плагины	через нативные модули
Защита от ошибок null	на уровне типов	нет	на уровне типов	нет
Асинхронные запросы	корутины	потоки, CompletableFuture	async/await	async/await, Promise

Kotlin выбран основным языком: только на нём пишется интерфейс Jetpack Compose, а корутины упрощают сетевые запросы. Java используется для моделей данных и утилит — оба языка компилируются в байт-код одной виртуальной машины и вызывают друг друга без ограничений. Flutter и React Native дали бы приложение и для iOS, но требуют отдельного языка и прослойки для доступа к возможностям Android, что задание не предусматривает.

Таблица 4 – Сравнение сред разработки

Критерий	Android Studio	IntelliJ IDEA	Visual Studio Code	Eclipse
Стоимость	бесплатно	Community — бесплатно	бесплатно	бесплатно
Эмулятор и менеджер устройств	встроены	через плагин Android	только расширения	поддержка Android прекращена
Предпросмотр Jetpack Compose	есть	через плагин Android	нет	нет
Gradle, Logcat, профилировщик	встроены	встроены	частично, расширениями	устарели

Android Studio — официальная среда Google на основе IntelliJ IDEA: эмулятор, предпросмотр Compose, сборка Gradle и отладчик доступны без дополнительной настройки и бесплатно.

Таблица 5 – Сравнение систем управления базами данных

Критерий	MS SQL Server	PostgreSQL	MySQL	Oracle Database
Лицензия	бесплатная редакция Express, платные Standard и Enterprise	бесплатная, открытый код	Community бесплатна, есть коммерческие	Express бесплатна, есть коммерческие
Язык процедур	T-SQL	PL/pgSQL	SQL/PSM	PL/SQL
Средство администрирования	SQL Server Management Studio	pgAdmin	MySQL Workbench	SQL Developer
Транзакции, внешние ключи, ограничения	да	да	да (InnoDB)	да

MS SQL Server задан исходными данными. Бесплатной редакции Express достаточно для базы приложения, а SQL Server Management Studio позволяет создавать таблицы, строить диаграмму базы данных и выполнять запросы. Транзакции и внешние ключи сохраняют целостность данных об арендах.

Таблица 6 – Библиотеки проекта и их аналоги

Библиотека	Назначение	Аналоги	Причина выбора
Jetpack Compose, Material 3	интерфейс и оформление	XML-разметка View	декларативный подход, рекомендация Google
Navigation Compose	переходы между экранами	Voyager, ручная навигация	официальная, типобезопасные маршруты
ViewModel, StateFlow	состояние экранов	LiveData	интеграция с Compose и корутинами
Retrofit, OkHttp, Gson	обмен с REST-сервером	Ktor Client, Volley	стандарт для Android, API задаётся интерфейсом
Coil 3	загрузка фотографий	Glide, Picasso	написана на Kotlin, поддерживает Compose
Yandex MapKit	карта с маркерами цен	Google Maps SDK, 2ГИС	подробные карты России, бесплатная версия
Kyant Backdrop	эффект «жидкого стекла»	Haze	размытие с преломлением и бликами

1.6 Принятые стандарты кодирования и план тестирования

Код оформлен по соглашениям Kotlin Coding Conventions и Java Code Conventions: классы — UpperCamelCase, функции и переменные — lowerCamelCase, константы — UPPER_SNAKE_CASE; экраны, модели представления и классы обмена с сервером получают суффиксы Screen, ViewModel и Dto; отступ — 4 пробела; денежные суммы хранятся в BigDecimal; ключи доступа в исходный код не записываются.

В соответствии с итеративной методологией тестирование проводилось в конце каждой итерации и включало:

1. функциональное тестирование по сценариям пункта 2.5 со сверкой цен;
2. негативное тестирование — ввод заведомо некорректных данных;
3. автоматический проход по экранам — тест ScreenshotTour;
4. контроль сборки APK в GitHub Actions после каждого изменения.

2. ПРАКТИЧЕСКИЙ РАЗДЕЛ

2.1 Требования к аппаратному и программному обеспечению

Требования к компьютеру разработчика и к устройству пользователя приведены в таблице 7.

Таблица 7 – Требования к аппаратному и программному обеспечению

Параметр	Компьютер разработчика	Устройство пользователя
Операционная система	Windows 10/11 (64 бит), macOS или Linux	Android 8.0 (API 26) и новее
Процессор	64-разрядный, от 4 ядер, с поддержкой виртуализации	ARM64, ARMv7 или x86_64
Оперативная память	от 8 ГБ, рекомендуется 16 ГБ	от 2 ГБ
Место на диске	от 16 ГБ для Android Studio, SDK и эмулятора	от 200 МБ
Программное обеспечение	Android Studio, Android SDK (API 37), Git, MS SQL Server и SSMS	доступ в интернет для карт и сервера

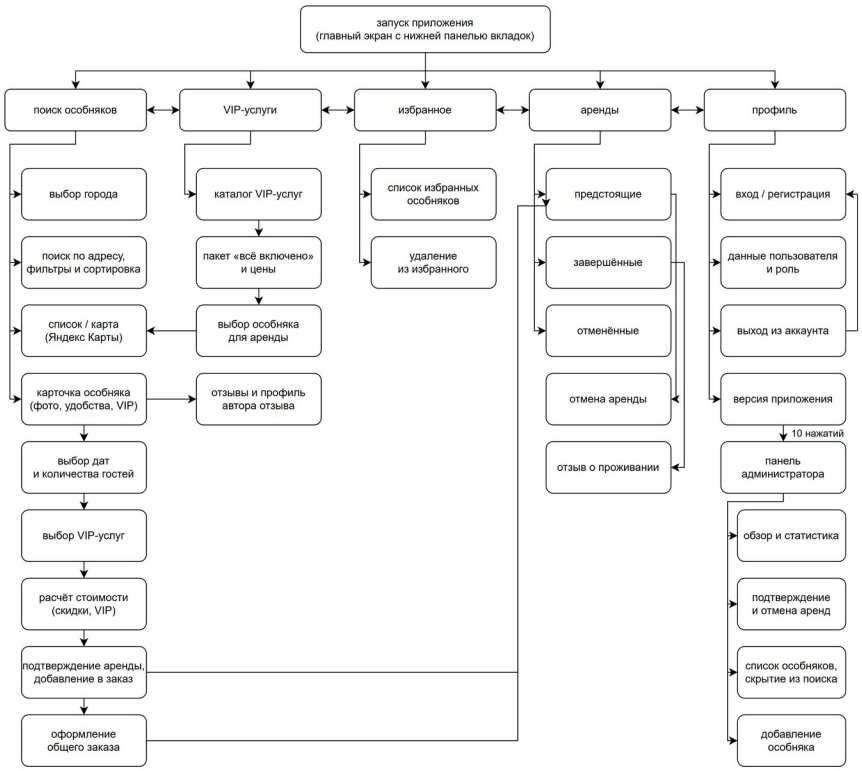
2.2 Описание пользовательского интерфейса, основных алгоритмов и структур данных

Пользовательский интерфейс построен на Jetpack Compose (Material 3) как набор экранов, связанных навигационным графом. Навигация построена на нижней панели из пяти вкладок: «Поиск», «VIP», «Избранное», «Аренды» и «Профиль». Пользователю доступны:

- поиск — выбор города, поиск по адресу и названию, фильтры по типу, датам и числу гостей, сортировка, переключение между списком и картой Яндекса с маркерами цен;
- карточка особняка — фотографии, описание, удобства, VIP-услуги, мини-карта и отзывы с переходом в профиль автора отзыва;
- оформление аренды — выбор дат по календарю занятости, числа гостей и VIP-услуг, расчёт стоимости со скидками;
- общий заказ — особняки добавляются в заказ, который открывается кнопкой «Заказ: N» и оформляется одним действием;
- «Аренды» — вкладки «Предстоящие», «Завершённые» и «Отменённые», отмена аренды и отзыв о проживании;
- «Избранное» и «Профиль» — сохранённые особняки, данные пользователя, вход, регистрация и выход;
- панель администратора — статистика, подтверждение и отмена аренд, добавление и скрытие особняков.

Отдельной вкладки для заказа нет — кнопка «Заказ: N» появляется, когда в нём есть хотя бы один особняк. Панель администратора не отображается в меню: по аналогии со скрытыми настройками мессенджера Telegram она открывается десятью нажатиями на строку «Версия приложения» в профиле и только для

учётной записи с ролью администратора. Структура разделов и экранов приведена на рисунке 3; выход из аккаунта возвращает пользователя к экрану входа.



Без входа в аккаунт «избранное», «аренды», «профиль» и оформление аренды открывают экран «вход / регистрация». Панель администратора в меню не показывается: она открывается 10 нажатиями на версию приложения в профиле и только для аккаунта администратора.

Рисунок 3 – Структура разделов и экранов приложения

Общий алгоритм работы приложения показан на рисунке 4.

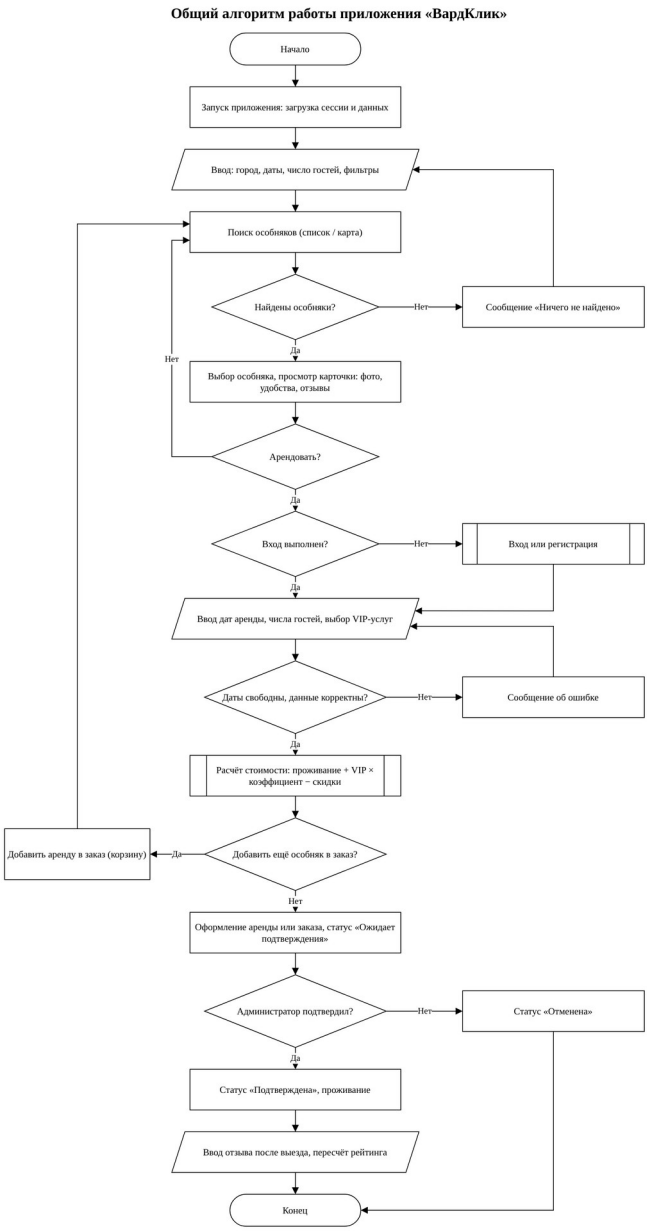


Рисунок 4 – Блок-схема общего алгоритма работы приложения

Основной алгоритм — расчёт стоимости аренды (объект Pricing): стоимость проживания равна цене за ночь, умноженной на число ночей; цена VIP-услуги равна базовой цене, умноженной на коэффициент класса особняка (с округлением до 500 ₽), посуточные услуги умножаются на число ночей; при выборе всех доступных услуг действует скидка 15 % на услуги и 5 % на проживание, в заказе из двух и более особняков — 7 % на проживание; итог равен сумме проживания и услуг за вычетом скидок. Перед созданием аренды сервер проверяет, что дата заезда не раньше текущей, срок — от 1 до 30 ночей, число гостей не больше вместимости, а даты не пересекаются с арендами в статусах «ожидает подтверждения» и «подтверждена»: две аренды пересекаются, если заезд каждой из них раньше выезда другой.

Основные структуры данных: классы моделей (особняк, аренда, отзыв,

VIP-услуга и др.), неизменяемые состояния экранов, которые ViewModel публикует через StateFlow, и запечатанный класс ApiResult с вариантами «успех», «ошибка» и «исключение».

2.3 Реализация программных модулей и описание их интеграции

Исходный код организован по пакетам: data.model — 18 классов моделей данных на Java; data.remote — интерфейс ApiService (24 метода REST API), клиент Retrofit с перехватчиком, добавляющим токен в заголовок запроса, и демонстрационный сервер MockApiService; data.repository — шесть репозиториев и функция safeApiCall; di — контейнер зависимостей AppContainer; ui — экраны 11 разделов, компоненты, навигация и тема; util — расчёт цен, проверка ввода и работа с датами.

Модуль взаимодействия с базой данных построен так: приложение не подключается к MS SQL Server напрямую, а обращается по HTTP к REST-серверу, который выполняет запросы к базе. Флаг сборки USE_MOCK переключает источник данных на демонстрационный сервер, хранящий те же данные (8 городов, 22 особняка, 237 отзывов, 6 VIP-услуг) на устройстве. База данных состоит из 13 связанных таблиц (рисунок 5).

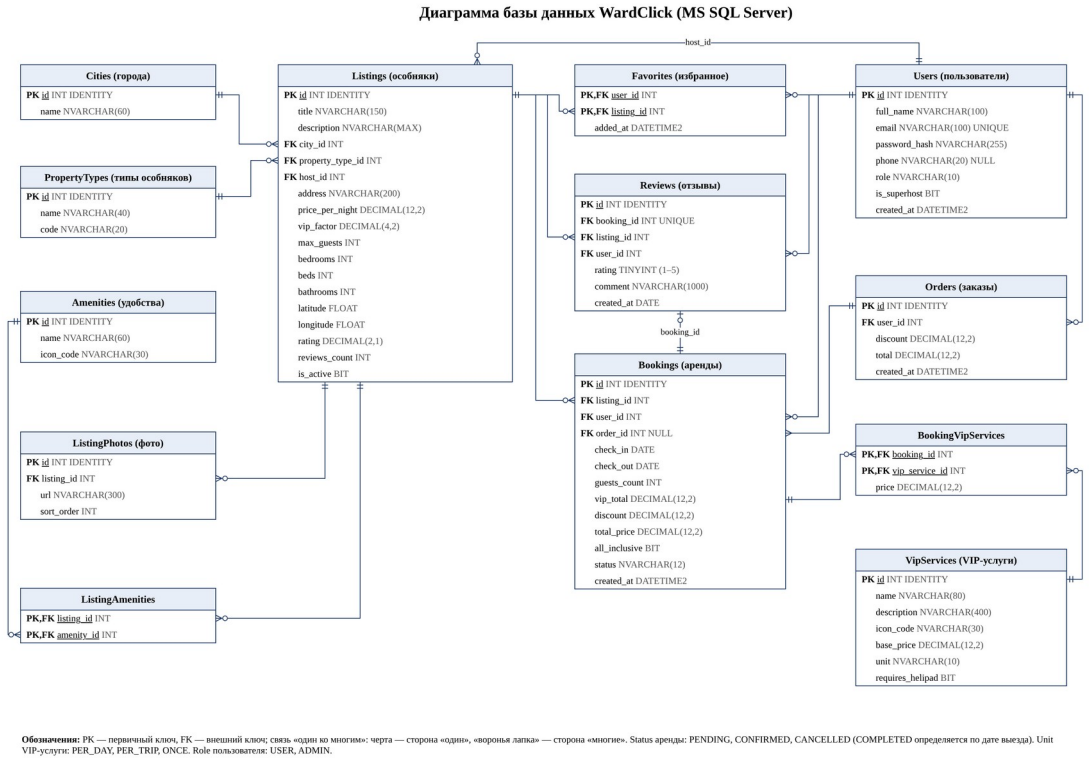
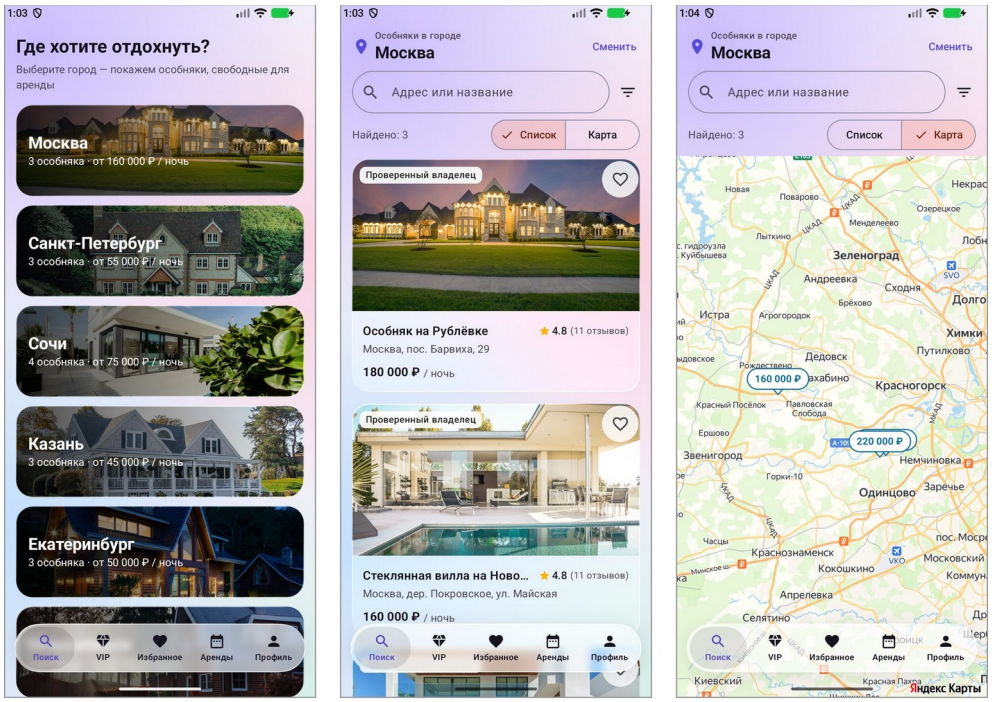


Рисунок 5 – Диаграмма базы данных

Внешний вид приложения выполнен в стиле «жидкого стекла»: полупрозрачные панели и нижнее меню размывают фотографии особняков под собой, акцентный цвет — золотой, поддерживаются светлая и тёмная темы.

Главный экран поиска (рисунок 6) содержит выбор города, список особняков с ценой за ночь и рейтингом и переключатель на карту с маркерами цен; остальные экраны приведены в приложении А.



а) б) в)

Рисунок 6 – Поиск: а) выбор города; б) список особняков; в) карта с маркерами цен

Карточка особняка (рисунок А.1) открывается из списка или с карты: галерея фотографий, описание, удобства и VIP-услуги с ценами, пересчитанными для класса особняка; фильтры позволяют отобрать особняки по цене, числу гостей, спален и рейтингу. Оформление аренды (рисунок А.2) выполняется по календарю, где занятые даты недоступны, после выбора числа гостей и VIP-услуг итоговая сумма пересчитывается сразу. Раздел отзывов с расположением особняка и каталог VIP-услуг показаны на рисунке А.3, экран входа и личные разделы — на рисунке А.4, профиль и панель администратора — на рисунке А.5. Ключевые фрагменты кода модулей приведены в приложении Б.

2.4 Обработка входных данных, исключительных ситуаций и ошибок

Входные данные проверяются дважды: на устройстве класс Validators проверяет email, длину пароля и телефон, а календарь не даёт выбрать прошедшие и занятые даты; на сервере все правила проверяются повторно. Запросы выполняются через функцию safeApiCall, которая перехватывает исключения и возвращает понятное сообщение (таблица 8).

Таблица 8 – Обработка ошибок и исключительных ситуаций

Ситуация	Код или исключение	Сообщение пользователю
Неверный email или пароль	401	«Неверный email или пароль»
Даты заняты другой арендой	409	«…: выбранные даты уже заняты»
Заезд в прошлом, больше 30 ночей, гостей больше вместимости	400	сообщение с причиной ошибки
Нет прав администратора	403	«Доступ запрещен»
Нет подключения к интернету	IOException	«Отсутствует подключение к интернету»
Сервер не ответил за 10 секунд	TimeoutCancellation Exception	«Время ожидания истекло»

2.5 Разработка тестовых наборов и тестовых сценариев

Тестирование проводилось в демонстрационном режиме: бизнес-логика проверялась на классах MockApiService и Pricing, интерфейс — на эмуляторе Android 14. Сценарии приведены в таблице 9.

Таблица 9 – Тестовые сценарии

№	Исходные данные	Действия	Ожидаемый результат	Фактический результат
1	«Особняк на Рублёвке» (180 000 ₽/ночь, коэффициент 1,5), 10.12–14.12.2026, 2 гостя, водитель и шеф-повар	оформить аренду	720 000 + 150 000 + 120 000 = 990 000 ₽, статус «Ожидает подтверждения»	990 000 ₽, статус PENDING
2	тот же особняк, 20.12–22.12.2026, все 6 услуг	оформить аренду	«Всё включено»: услуги 819 000 ₽, скидка 122 850 + 18 000 = 140 850 ₽, итог 1 038 150 ₽	скидка 140 850 ₽, итог 1 038 150 ₽
3	тот же особняк, 12.12–16.12.2026	оформить аренду на занятые даты	аренда не создана, ошибка 409 «выбранные даты уже заняты»	соответствует ожидаемому
4	заказ: «Особняк на Рублёвке» и «Особняк на Крестовском острове», по 2 ночи	оформить общий заказ	скидка 7 % на проживание: 25 200 и 21 000 ₽, один номер заказа	скидки 25 200 и 21 000 ₽, общий заказ
5	пользователь user; затем администратор admin	запросить статистику; подтвердить аренду из сценария 1	пользователю — ошибка 403; администратору — статус «Подтверждена»	403; статус CONFIRMED

Дополнительно проверены отказ при неверном пароле (401), заезде в прошлом, превышении вместимости и срока 30 ночей (400) и повторная отмена аренды (409): во всех случаях получена ожидаемая ошибка.

2.6 Анализ результатов тестирования и описание исправленных дефектов

Все сценарии выполнены, фактические результаты совпали с ожидаемыми; автоматизированный тест ScreenshotTour проходит основные экраны без сбоев в светлой и тёмной темах и на узком экране. Дефекты, найденные и исправленные в ходе разработки, приведены в таблице 10.

Таблица 10 – Исправленные дефекты

№	Дефект	Причина	Исправление
1	вкладка «Поиск» открывала раздел «Аренды»	восстанавливался сохранённый стек другой вкладки	возврат к корню стека навигации
2	в карточке 23 отзыва, а в списке только 2	число отзывов и рейтинг были заданы вручную	расчёт по самим отзывам
3	особняк «в лесу» на карте оказывался в море	координаты не соответствовали адресу	координаты сверены с картой
4	серые пятна под полупрозрачными карточками	системная тень просвечивала сквозь стекло	тень только снаружи формы
5	сжатые кнопки «Отменить» и «Подтвердить»	кнопки в одной строке с ценой	отдельная строка равной ширины

2.7 Инспектирование компонентов программы по чек-листу

Исходный код проверен по чек-листу соблюдения стандартов кодирования (приложение В): именование, структура пакетов, асинхронность, обработка ошибок, отсутствие ключей доступа в коде. Нарушений, влияющих на работу приложения, не выявлено.

2.8 Краткая инструкция пользователя

В приложении предусмотрены три роли доступа:

- **гость** — без входа просматривает каталог, карточки особняков, отзывы, карту и VIP-услуги; при попытке арендовать особняк или открыть избранное, аренды и профиль приложение предлагает войти;
- **пользователь** — после входа или регистрации арендует особняки, формирует общий заказ, ведёт избранное, отменяет предстоящие аренды и оставляет отзывы о завершённом проживании;
- **администратор** — дополнительно открывает панель администратора десятью нажатиями на «Версию приложения» в профиле, подтверждает или отменяет заявки, добавляет и скрывает особняки.

Чтобы арендовать особняк, выберите город на вкладке «Поиск», откройте карточку особняка, нажмите «Арендовать», выберите даты, число гостей и VIP-услуги и подтвердите аренду. Кнопка «В заказ» добавляет особняк в общий заказ, а статус аренды отображается в разделе «Аренды».

3. ОХРАНА ТРУДА

Разработка программного обеспечения связана с длительной работой за персональным компьютером. Чтобы сохранить здоровье и работоспособность программиста, необходимо учитывать опасные и вредные производственные факторы и выполнять требования к организации рабочего места, установленные Трудовым кодексом Российской Федерации, СанПиН 1.2.3685-21, СП 2.2.3670-20 и стандартами системы безопасности труда.

3.1 Опасные и вредные производственные факторы

В соответствии с ГОСТ 12.0.003-2015 при работе за компьютером на программиста воздействуют следующие факторы:

- физические — опасность поражения электрическим током, статическое электричество, недостаточная освещённость рабочей зоны, прямая и отражённая блёсткость экрана, повышенный уровень шума от вентиляторов и оргтехники, неблагоприятный микроклимат;
- психофизиологические — длительные статические нагрузки при работе сидя, перенапряжение зрения, умственное перенапряжение, монотонность труда и эмоциональные перегрузки при сжатых сроках.

Длительное воздействие этих факторов приводит к утомлению глаз («компьютерному зрительному синдрому»), болям в шее и спине, туннельному синдрому запястья, головным болям и снижению внимания, поэтому рабочее место и режим труда организуются так, чтобы свести их влияние к минимуму.

3.2 Требования к организации рабочего места и освещению

Площадь на одно рабочее место с компьютером на базе плоского монитора должна составлять не менее 4,5 м². Монитор располагается на расстоянии 60–70 см от глаз, но не ближе 50 см, верхний край экрана — на уровне глаз или немного ниже. Высота рабочей поверхности стола — 680–800 мм, под столом должно быть свободное место для ног. Рабочий стул должен быть подъёмно-поворотным, регулироваться по высоте, наклону спинки и иметь подлокотники; при необходимости используется подставка для ног. Клавиатура располагается на расстоянии 10–30 см от края стола, чтобы запястья не находились на весу.

Помещение должно иметь естественное и искусственное освещение. Окна располагаются сбоку от рабочего места, предпочтительно слева, и оборудуются жалюзи. Освещённость поверхности стола в зоне работы с документами должна составлять 300–500 лк, светильники не должны давать бликов на экране, а коэффициент пульсации освещённости не должен превышать 5 %. Для помещений с компьютерами поддерживаются оптимальные параметры

микроклимата для лёгкой работы: температура воздуха 22–24 °С в холодный и 23–25 °С в тёплый период года, относительная влажность 40–60 %; уровень шума на рабочем месте программиста не должен превышать 50 дБА.

Монитор должен иметь антибликовое покрытие и регулировку яркости и контраста; яркость экрана подбирается близкой к яркости окружающих предметов, а размер шрифта и масштаб — такими, чтобы текст читался без напряжения. Экран и клавиатуру регулярно очищают от пыли, а помещение ежедневно проветривают и убирают влажным способом.

3.3 Режим труда и отдыха

Продолжительность непрерывной работы за компьютером без перерыва не должна превышать 2 часов. Рекомендуется делать перерывы по 10–15 минут через каждые 1–1,5 часа работы и выполнять в это время гимнастику для глаз, упражнения для шеи, плеч и спины. Полезно чередовать работу с кодом и задачи, не связанные с экраном: обсуждение требований, работу с документацией. Перерывы и гимнастика снижают утомление и повышают производительность труда, поэтому их следует соблюдать и при работе с жёсткими сроками. Работники, занятые на работах с компьютером, проходят периодические медицинские осмотры, а при появлении усталости глаз или болей в спине режим работы и организацию рабочего места пересматривают.

3.4 Безопасная эксплуатация оборудования

Компьютер является электроустановкой напряжением 220 В, поэтому его подключают только к исправным заземлённым розеткам через сетевой фильтр или источник бесперебойного питания. Запрещается работать с повреждёнными кабелями и вилками, вскрывать корпус включённого оборудования, ставить рядом с ним ёмкости с жидкостью и перегружать розетки удлинителями. Перед началом работы сотрудник проходит вводный и первичный инструктаж по охране труда и получает I группу по электробезопасности.

Помещение оборудуется углекислотным огнетушителем, пригодным для тушения электроустановок под напряжением, и планом эвакуации. При появлении запаха гари, дыма или искрения необходимо отключить оборудование от сети, сообщить руководителю и при необходимости вызвать пожарную охрану по номеру 101 или 112. Соблюдение перечисленных требований обеспечивает безопасные условия труда при разработке и тестировании приложения «ВардКлик».

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта поставленная цель достигнута: спроектировано, программно реализовано и протестировано мобильное приложение «ВардКлик» для бронирования загородных особняков и VIP-услуг на платформе Android. Задачи, сформулированные во введении, решены.

Анализ предметной области показал, что аренда особняков сегодня организуется вручную: нет единого графика занятости, цены дополнительных услуг непрозрачны, а для аренды нескольких домов приходится договариваться с каждым владельцем. Сравнение с сервисами Airbnb, Суточно.ру, Островок, Яндекс Путешествия и Авито Путешествия подтвердило, что ни один из них не объединяет аренду премиального особняка, заказ VIP-услуг с ценами по классу объекта и единый заказ на несколько особняков.

Сформулированы функциональные и нефункциональные требования, спроектированы диаграмма классов, диаграмма состояний, база данных MS SQL Server из 13 таблиц и блок-схема общего алгоритма. По шаблону MVVM реализован интерфейс на Jetpack Compose: каталог с фильтрами и картой Яндекса, карточка особняка, оформление аренды и общего заказа, личные разделы и панель администратора. Реализована бизнес-логика: проверка занятости дат, цены VIP-услуг по коэффициенту класса особняка, скидки за пакет «Всё включено» (15 % на услуги и 5 % на проживание) и за заказ нескольких особняков (7 %).

Модуль взаимодействия с данными работает через REST-сервер и поддерживает демонстрационный режим. Тестирование по сценариям и автоматизированный проход по экранам подтвердили корректность работы, найденные дефекты исправлены. Рассмотрены вопросы охраны труда и организации рабочего места программиста. Итеративная методология позволила наращивать функциональность и проверять каждую версию на эмуляторе.

Практическая значимость работы состоит в том, что приложение позволяет клиенту в одном месте подобрать загородный особняк, заказать к нему услуги с прозрачной итоговой ценой и оформить заказ на несколько объектов, а администратору — вести каталог и подтверждать заявки. В дальнейшем можно добавить онлайн-оплату, push-уведомления о смене статуса аренды и опубликовать приложение в RuStore.

СПИСОК ИСТОЧНИКОВ ИНФОРМАЦИИ

Основные источники

1. Жемеров, Д. Kotlin в действии / Д. Жемеров, С. Исакова ; пер. с англ. — Москва : ДМК Пресс, 2018. — 402 с.

2. Филлипс, Б. Android. Программирование для профессионалов / Б. Филлипс, К. Стюарт, К. Марсикано, Б. Гарднер. — 4-е изд. — Санкт-Петербург : Питер, 2021. — 704 с.

3. Мартин, Р. Чистый код: создание, анализ и рефакторинг / Р. Мартин. — Санкт-Петербург : Питер, 2019. — 464 с.

4. Бен-Ган, И. Microsoft SQL Server 2012. Основы T-SQL / И. Бен-Ган. — Москва : Эксмо, 2015. — 400 с.

5. Фаулер, М. UML. Основы. Краткое руководство по стандартному языку объектного моделирования / М. Фаулер. — 3-е изд. — Санкт-Петербург : Символ-Плюс, 2011. — 192 с.

6. ГОСТ 19.701-90. ЕСПД. Схемы алгоритмов, программ, данных и систем. — Москва : Стандартинформ, 2010. — 23 с.

7. ГОСТ 12.0.003-2015. ССБТ. Опасные и вредные производственные факторы. Классификация. — Москва : Стандартинформ, 2019. — 10 с.

8. СанПиН 1.2.3685-21. Гигиенические нормативы и требования к обеспечению безопасности и (или) безвредности для человека факторов среды обитания. — Москва, 2021.

9. СП 2.2.3670-20. Санитарно-эпидемиологические требования к условиям труда. — Москва, 2020.

Интернет-ресурсы

10. Jetpack Compose : документация Android Developers. — URL: https://developer.android.com/compose (дата обращения: 04.11.2026).

11. Guide to app architecture. — URL: https://developer.android.com/topic/architecture (дата обращения: 04.11.2026).

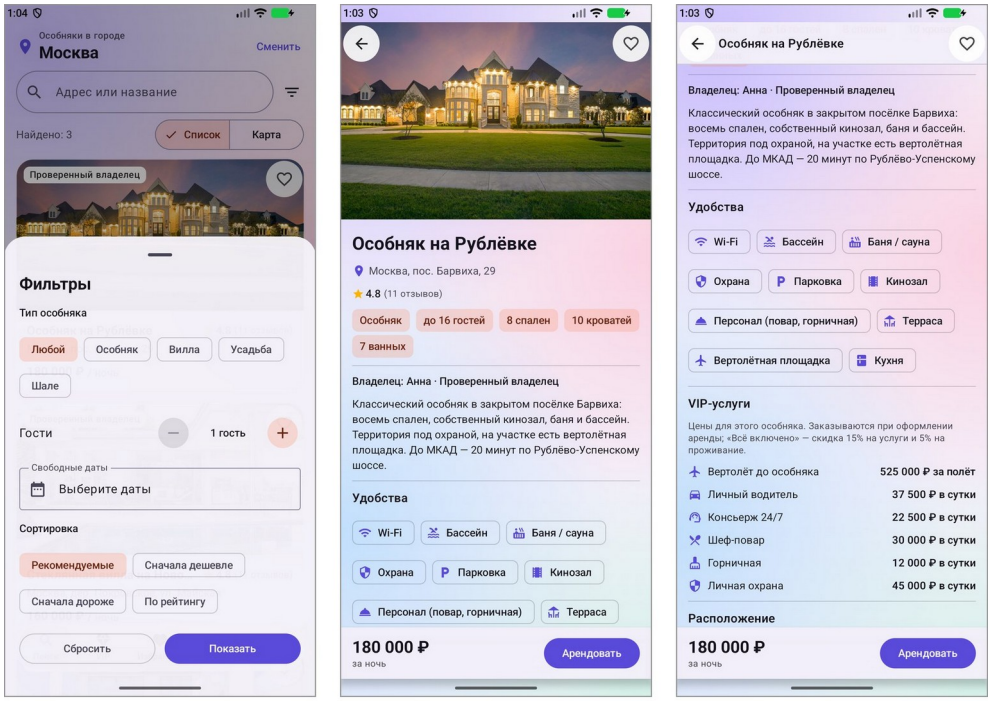
12. Kotlin coding conventions. — URL: https://kotlinlang.org/docs/coding-conventions.html (дата обращения: 04.11.2026).

13. Retrofit. — URL: https://square.github.io/retrofit/ (дата обращения: 04.11.2026).

14. MapKit SDK : Яндекс для разработчиков. — URL: https://yandex.ru/dev/mapkit/ (дата обращения: 04.11.2026).

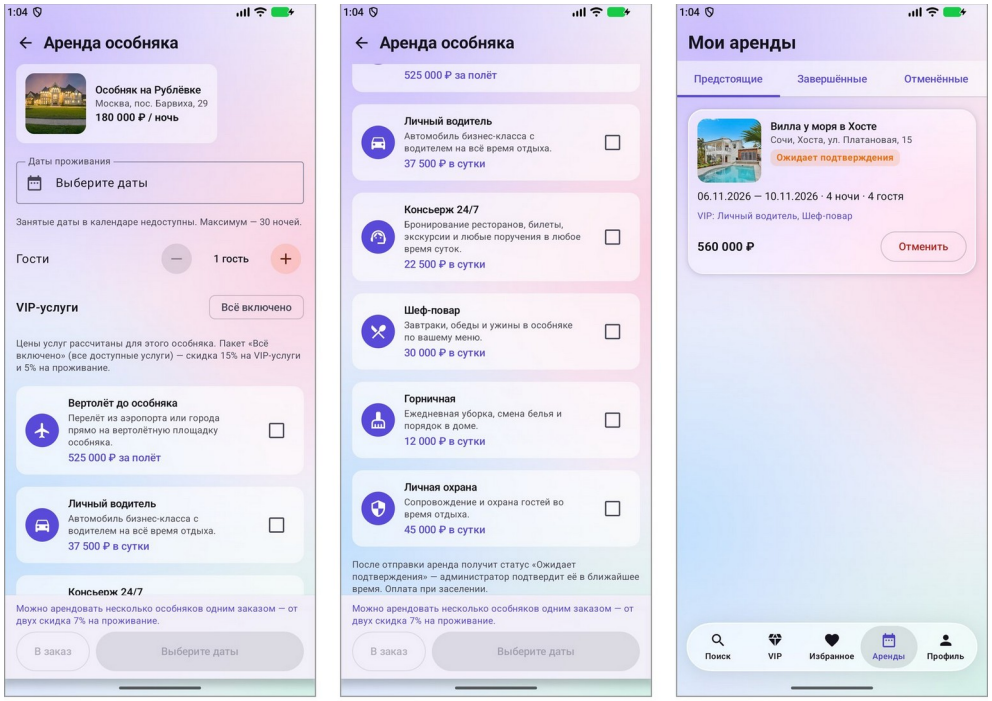
15. Документация по SQL Server : Microsoft Learn. — URL: https://learn.microsoft.com/ru-ru/sql/sql-server/ (дата обращения: 04.11.2026).

ПРИЛОЖЕНИЕ А
Экранные формы приложения



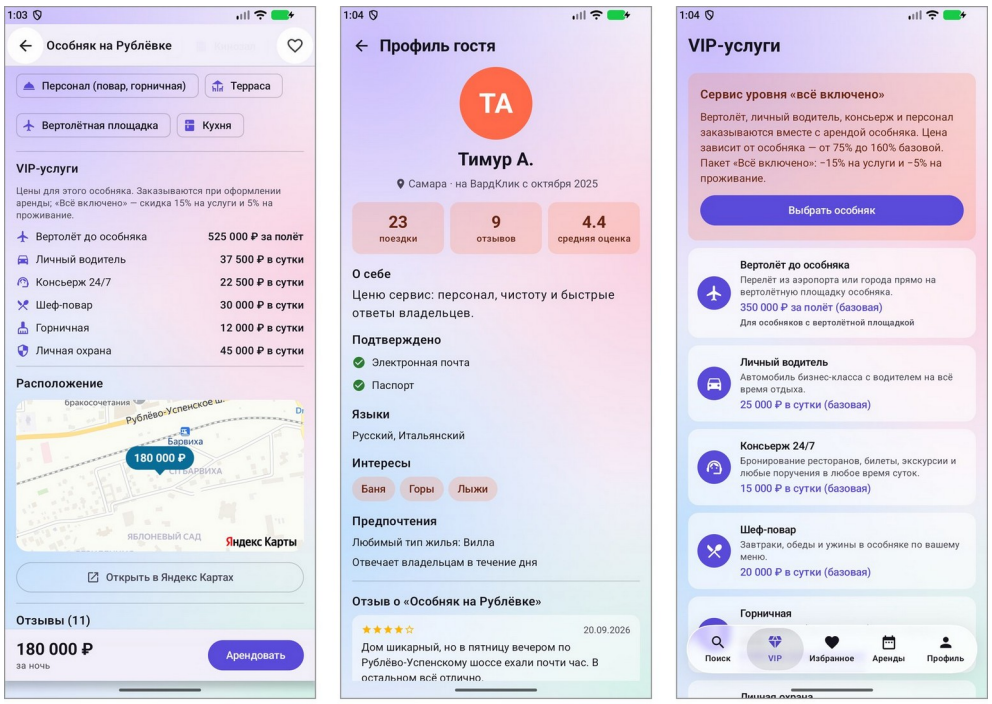
а) б) в)

Рисунок А.1 – Фильтры и карточка особняка: а) фильтры; б) карточка; в) удобства и VIP-услуги



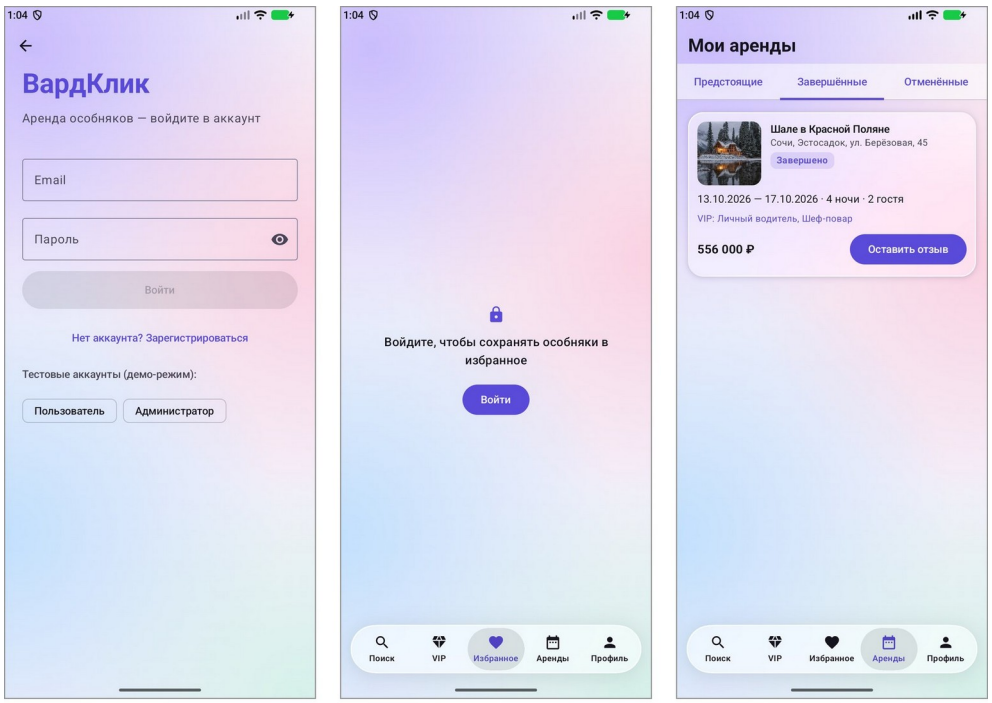
а) б) в)

Рисунок А.2 – Оформление аренды: а) даты и гости; б) VIP-услуги; в) предстоящие аренды



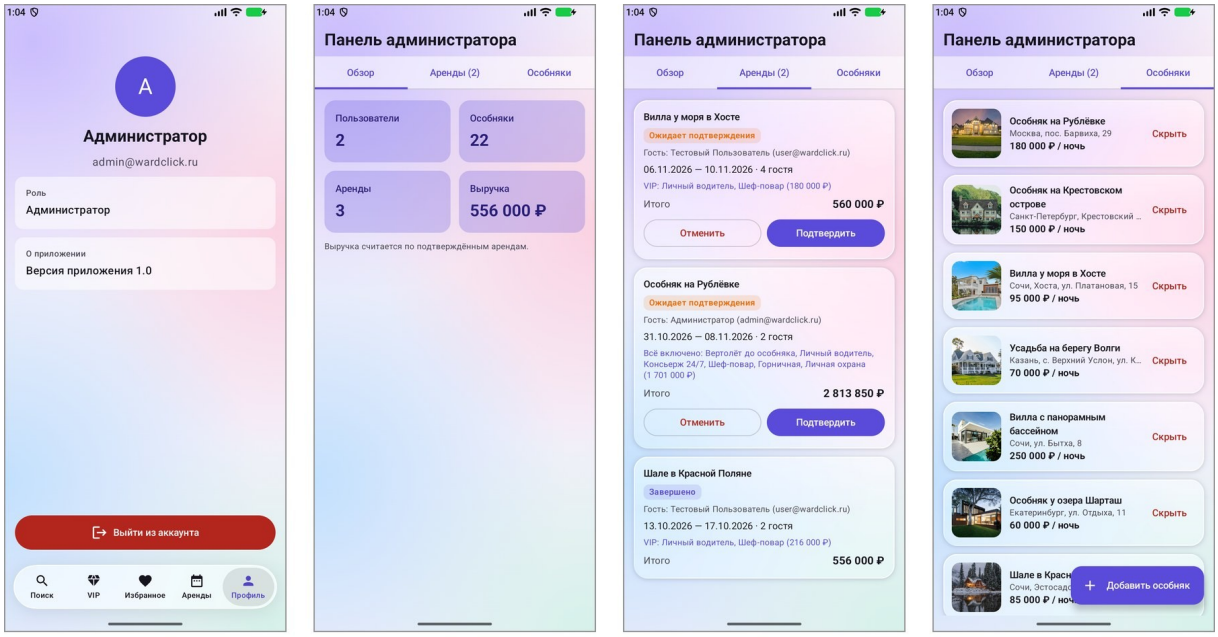
а) б) в)

Рисунок А.3 – Отзывы и VIP-услуги: а) расположение и отзывы; б) профиль автора отзыва; в) каталог VIP-услуг



а) б) в)

Рисунок А.4 – Вход и личные разделы: а) вход; б) избранное без входа; в) завершённые аренды и отзыв



а) б) в) г)

Рисунок А.5 – Профиль и панель администратора: а) профиль; б) обзор; в) подтверждение аренд; г) управление особняками

ПРИЛОЖЕНИЕ Б
Фрагменты исходного кода

Листинг Б.1 – Расчёт стоимости аренды (util/Pricing.kt)

```
fun quote(listing: ListingDetailsDto, nights: Long, chosen: List<VipServiceDto>,
          available: List<VipServiceDto>, multi: Boolean): Quote {
    // проживание: цена за ночь × число ночей
    val stay = (listing.pricePerNight ?:
BigDecimal.ZERO).multiply(BigDecimal.valueOf(nights))
    // услуги: цена уже умножена на коэффициент особняка, посуточные — × ночи
    val vip = chosen.fold(BigDecimal.ZERO) { acc, s -> acc.add(s.costFor(nights)) }
    val chosenIds = chosen.map { it.id }.toSet()
    // «Всё включено» — выбраны все услуги, доступные в особняке
    val allInclusive = available.isNotEmpty() && available.all { it.id in chosenIds }
    val aiDiscount = if (allInclusive) {
        percent(vip, ALL_INCLUSIVE_VIP_PERCENT).add(percent(stay,
ALL_INCLUSIVE_STAY_PERCENT))
    } else BigDecimal.ZERO
    // заказ из двух и более особняков: −7 % на проживание
    val multiDiscount = if (multi) percent(stay, MULTI_STAY_PERCENT) else BigDecimal.ZERO
    return Quote(stay, vip, allInclusive, aiDiscount, multiDiscount)
}
```

Листинг Б.2 – Проверка заявки на аренду (data/remote/MockApiService.kt)

```
if (inD.isBefore(LocalDate.now()))
    throw ApiException(400, "Дата заезда не может быть в прошлом")
val nights = ChronoUnit.DAYS.between(inD, outD)
if (nights <= 0) throw ApiException(400, "Дата выезда должна быть позже даты заезда")
if (nights > MAX_NIGHTS) throw ApiException(400, "Максимальный срок аренды — $MAX_NIGHTS
ночей")
if (request.guestsCount < 1 || request.guestsCount > listing.maxGuests)
    throw ApiException(400, "${listing.title}: количество гостей — от 1 до $
{listing.maxGuests}")
if (overlaps(listing.id, inD, outD))
    throw ApiException(409, "«${listing.title}»: выбранные даты уже заняты")

// пересечение с действующими арендами (ожидает подтверждения или подтверждена)
private fun overlaps(listingId: String, inD: LocalDate, outD: LocalDate) =
    state.bookings.any { b ->
        b.listingId == listingId && (b.status == "PENDING" || b.status == "CONFIRMED") &&
            inD.isBefore(parseDate(b.checkOutDate)) &&
outD.isAfter(parseDate(b.checkInDate))
    }
```

Листинг Б.3 – Обработка ошибок запросов (data/repository/ApiResult.kt, фрагмент)

```
suspend fun <T> safeApiCall(timeoutMs: Long = 10000L, apiCall: suspend () -> T):
ApiResult<T> {
    return try {
        withTimeout(timeoutMs) { ApiResult.Success(apiCall()) }
    } catch (e: CancellationException) {
        if (e is TimeoutCancellationException) ApiResult.Exception(e, "Время ожидания
истекло")
        else throw e
    } catch (e: ApiException) {
        ApiResult.Error(e.code, e.message ?: "Ошибка")
    } catch (e: HttpException) {
        // ... errorMessage: текст из ответа сервера или сообщение по коду (400, 401,
403, 404, 409, 5xx)
        ApiResult.Error(e.code(), errorMessage)
    } catch (e: IOException) {
        ApiResult.Exception(e, "Отсутствует подключение к интернету")
    }
}
```

Листинг Б.4 – Скрытый вход в панель администратора (ui/screens/profile/ProfileScreen.kt)

```
// 10 нажатий на версию приложения; обычному пользователю нажатия ничего не дают
Card(modifier = Modifier.fillMaxWidth().clickable {
    if (!p.isAdmin) return@clickable
    taps++
    val left = ADMIN_TAPS - taps
    when {
        left <= 0 -> { taps = 0; onOpenAdmin() }
        left <= 4 -> Toast.makeText(context, "Ещё $left — и откроется панель
администратора",
                                    Toast.LENGTH_SHORT).show()
    }
}) { /* «Версия приложения 1.0» */ }
```

ПРИЛОЖЕНИЕ В
Чек-лист инспектирования кода

Таблица В.1 – Чек-лист инспектирования кода

№	Проверяемое требование	Результат
1	Имена классов, функций, переменных и констант соответствуют соглашениям Kotlin и Java	выполнено
2	Пакеты соответствуют слоям архитектуры (data, di, ui, util)	выполнено
3	Экраны не обращаются к сети напрямую — только через ViewModel и репозитории	выполнено
4	Сетевые запросы и обращения к данным выполняются асинхронно в корутинах	выполнено
5	Ошибки запросов перехватываются и показываются пользователю понятным сообщением	выполнено
6	Входные данные проверяются на устройстве и повторно на сервере	выполнено
7	Денежные суммы хранятся и рассчитываются в BigDecimal	выполнено
8	Ключ Яндекс Карт и другие секреты не хранятся в исходном коде и репозитории	выполнено
9	Состояние экранов хранится в ViewModel и переживает поворот экрана	выполнено
10	Тексты интерфейса на русском языке, интерфейс работает в светлой и тёмной темах	выполнено
11	Неочевидные решения и бизнес-правила прокомментированы	выполнено
12	Проект собирается без ошибок в GitHub Actions после каждого изменения	выполнено